What Was Conceptually New in SHACL?

A Calibrated Re-examination against RacerPro nRQL, Negation as Failure, and the Data Substrate

Revised edition, with independent literature grounding

Prepared from the RacerPro manuals, the SHACL specifications,
and primary sources in the description-logic and RDF-validation literature

Drafted by a generative AI system (Claude Opus 4.8)
and substantiated against cited publications; see the authorship declaration.

June 2, 2026

Abstract

This report re-examines a specific historical-semantic question: what did SHACL make expressible as a constraint language that was not already expressible, in principle, in RacerPro via nRQL, negation as failure (NAF), projection, rules, concrete-domain/datatype predicates, and the substrate / OWL data-mirroring layer? It revises an earlier draft in four ways. First, it grounds the priority discussion in the description-logic literature rather than in the vendor manual alone: the idea of querying "what is known" through an epistemic / minimal-knowledge operator was formalized by Donini et al. in 1998 [5], and the "effective first-order query" agenda was crystallized by EQL-Lite in 2007 [7]; nRQL (2004–2005) [3, 4] sits between them as, defensibly, the first *practical, high-performance* engine to offer negation as failure plus projection over an OWL knowledge base. Contemporary RDF query languages such as RQL (2002) [8] and RDQL (January 2004) [9] did not offer negated projected subqueries (RDQL admitted only "safe" negation with no variables in the negated clause). This supports a *prior-art* and *conceptual-anticipation* claim, but not a claim of *documented influence* on SPARQL 1.1, SHACL, or property-graph languages, and not a claim of global "firstness."

Second, it replaces the earlier "informal expressivity theorem"—which assumed away exactly the hard cases—with an explicit *reduction lemma* whose stated assumptions mark the true boundary of the comparison. Third, it sharpens what is genuinely new in SHACL to two items with rigorous backing rather than one: (i) a *recursive* shape-conformance relation, whose semantics is a nontrivial object (partial / stable / well-founded assignments) and whose associated decision problems can be NP-hard or undecidable [16, 17, 19]; and (ii) *unbounded regular property paths*, which are provably beyond the nonrecursive negation as failure-plus-projection fragment because transitive closure is not first-order definable [20, 21]. RDF-term-native components (node kind, language tags, lexical form) and SHACL's validation-report vocabulary are conditional or organizational differences. Fourth, it adds a section on three *semantic traps* in the naive translation—epistemic-mode mismatch, cardinality counting without a unique-name assumption, and the active-domain-versus-anonymous-witness gap—that the earlier draft underweighted.

The net conclusion is unchanged in spirit but more precise: the large majority of recurring *OWL-centric, finite, closed-world* integrity checks that SHACL is used for were already within reach of nRQL plus the mirror substrate by 2005, and nRQL deserves explicit recognition as prior art for negated projected subqueries in a practical OWL query engine. In place of the earlier draft's bare "80%" we report an empirical *band* from coding the W3C SHACL test suite (§11): **73–84%** of SHACL Core constraint features have a direct nRQL counterpart ($\approx$92% once the substrate mirror is granted), while the *provably* native residual is only $\approx$5% (unbounded/recursive paths, matching Proposition 1), rising to a best estimate of $\approx$8% once `sh:pattern` regular expressions are counted—which RacerPro cannot express *declaratively*. Crucially, nRQL's lambda/`:reject` hatch is RacerPro's deliberately *termination-safe* "MiniLisp"—not Turing-complete (recursion is aborted at runtime; no regex engine), as we confirmed live on RacerPro 2.0—so, unlike SHACL's own (Turing-complete) SPARQL/JS hatches, it does *not* close the residual: unbounded paths and full-regex patterns survive it. That termination guarantee is the principled choice for a decidable DL query engine. We also *correct* the earlier draft's property-graph stance (§12): the data substrate is a genuine, reasoning-coupled property graph with first-class, queryable edge property maps—real prior art for property-graph modeling fused with DL reasoning—though the attributed-graph model class predates it (1984–1994) and we find no documented influence on later property-graph languages. The residual novelty of SHACL is concentrated, precisely, in recursive shape conformance, native regular-path validation, RDF-term primitives, and the standardized conformance/report model.

Generative AI authorship and independence declaration

This report was drafted by a generative AI system (Claude Opus 4.8). The human user supplied the research question, the RacerPro manuals, and a request to substantiate or disprove the priority claims using external sources. Those inputs were treated as material and hypotheses to be tested,

not conclusions to be adopted. Where the user's earlier draft proposed a figure ("80%"), a property-graph priority claim, or an "informal theorem," this revision either qualifies, rescopes, or withdraws the claim and says so explicitly. Bibliographic claims about dates and contents were checked against the cited publications; the description-logic and RDF-validation references below were located through literature search rather than supplied by the user. The report credits RacerPro/nRQL as strong prior art where the manuals and the dated publications support it, and it does not assert documented causal influence on later standards absent a citation trail.

Contents

1 Question, scope, and what changed

The question is conceptual, not sociological. It is *not* whether SHACL became a W3C standard, attracted more implementations, or integrated better with generic RDF tooling; those are real and important facts, and they are out of scope here. The question is:

> What did SHACL make expressible as a constraint language that was not already expressible, in principle, in RacerPro by nRQL plus the substrate representation layer, the mirror data substrate, and hybrid nRQL queries?

This revision differs from the earlier draft in four substantive ways, each justified in the body:

1. **Priority is grounded in the literature, not the manual alone** (§3). The manual's self-description ("the only practically available OWL query language of its time" for autoepistemic queries [1, p. 82]) is a vendor claim. We keep it, but we anchor the substantive priority claim in dated, independent publications and name the contemporaries it is implicitly measured against.
2. **The "informal theorem" becomes an explicit reduction** (§4) whose assumptions are the point.
3. **Two genuine novelties, with proofs/citations, not one** (§5): recursive shape conformance and unbounded regular paths.
4. **A new section on translation traps** (§6), **withdrawal of the "80%" figure** (§10), and a **corrected, evidence-based property-graph assessment** (§12).

Throughout, "possible with nRQL" is read generously, matching the system the manuals describe: ABox queries with classical negation, negation as failure, projection, union, and conjunction; DL-aware atoms evaluated by entailment; rules with query-body antecedents and generalized ABox consequents; concrete-domain/datatype predicates; group-by and aggregation; complex TBox queries; hybrid ABox-plus-substrate queries; and the mirror data substrate for OWL documents, annotations, and told datatype/annotation values [1, pp. 4, 81–84, 113–143]. On the SHACL side we distinguish SHACL Core, SHACL-SPARQL, and SHACL Advanced Features [12, 13].

2 Background, compressed

2.1 nRQL's epistemic profile

nRQL query bodies are built compositionally with `and`, `union`, `neg`, and `project-to` [1, pp. 81–82]. A *positive* atom binds a variable only to individuals for which the atom, after substitution, is *logically entailed* by the RacerPro knowledge base [1, p. 82]; positive atoms are therefore open-world and DL-aware. The `neg` constructor is negation as failure: it succeeds when the enclosed body has no answer over the active domain of *known* individuals (or substrate nodes) [1, pp. 113–117]. Crucially, `project-to` may appear *inside* a body, so negation as failure can be applied to a projected subquery—an anti-semi-join / `NOT EXISTS` pattern. This hybrid—open-world entailment for positive atoms, closed-world failure for selected subexpressions—is exactly the "autoepistemic" profile the manual advertises [1, p. 82], and it is the technical heart of the comparison.

2.2 The substrate and the mirror

A *data substrate* is a node- and edge-labeled directed graph whose labels are Lisp symbols, strings, numbers, and characters [1, p. 133]; a substrate may be associated with an ABox so that nRQL becomes a *hybrid* query language addressing both [1, pp. 133–137]. The *mirror data substrate* automatically materializes substrate objects for ABox individuals and, for an OWL KB, for OWL elements—adding markers such as `:owl-annotation`, `:owl-annotation-relationship`, `:owl-annotation-value`, `:owl-datatype-value`, and `:owl-datatype-role`—so that annotations and told datatype/annotation fillers become *queryable data* [1, pp. 137–143]. This is the single most under-appreciated fact for a fair comparison: it refutes the common claim that nRQL was "only" an ABox query language unable to reach ontology metadata.

2.3 SHACL, compressed

SHACL Core defines *shapes* (node and property shapes) with constraint components (`sh:minCount`, `sh:maxCount`, `sh:class`, `sh:datatype`, `sh:nodeKind`, `sh:pattern`, `sh:languageIn`, `sh:closed`, logical combinators, qualified value shapes, property paths). Its primary semantic object is a *conformance* relation between a data graph and a shapes graph, with structured *validation results* for non-conformances [12]. SHACL-SPARQL allows constraints written as SPARQL queries; SHACL Advanced Features adds rules [13]. SHACL's visible design lineage runs through SPIN [15], OSLC Resource Shapes, and Shape Expressions (ShEx) [14], not through DL query languages.

3 Historical priority, done carefully

3.1 Three claims, kept separate

Following standard practice we separate:

1. **Prior art:** nRQL had a feature before a later language standardized or popularized a similar one.
2. **Conceptual convergence:** two communities independently built similar mechanisms because they faced the same finite-graph constraint problem.
3. **Direct influence:** designers of the later language knew of, cited, or built on nRQL.

Only the first is strongly supported here; the second is plausible; the third we do not assert.

3.2 What the idea owes to earlier DL work

Intellectual honesty requires noting that nRQL did not invent the *idea* of asking a DL knowledge base "what is known." Donini, Lenzerini, Nardi, Nutt and Schaerf formalized an *epistemic operator* **K** for description logics in 1998, explicitly to capture queries, procedural rules, and closed-world reasoning over an open-world KB [5]; the minimal-knowledge / negation as failure semantics was further developed in the MKNF line [6]. nRQL's contribution is therefore not the concept of autoepistemic querying but its *practical realization*: a high-performance engine in which negation as failure, projection, aggregation, rules, and concrete-domain predicates are available together over real OWL data, benchmarked on LUBM [3, 4]. Read this way, the priority claim becomes both more modest and more defensible.

3.3 What the dated record supports

The nRQL idiom that matters is `NEG` over a projected (or named) query body:

Listing 1: nRQL negated projected query body: an anti-existential subquery.

```
(retrieve (?x)
  (neg (project-to (?x)
         (and (?x c) (?x ?y r) (?y d)))))
```

Listing 2: NAF applied to a defined query body.

```
(defquery mother-of (?x ?y)
  (and (?x woman) (?x ?y has-child)))
(retrieve (?a ?b)
  (neg (substitute (mother-of ?a ?b))))
```

These appear in the user's guide [1, pp. 119–122] and the mechanism is described in the dated 2004–2005 publications [3, 4]. The relevant comparison dates are: RQL, 2002 [8]; RDQL, W3C Member Submission of 9 January 2004 [9]; nRQL, ADL'04 (September 2004) and DL 2005 [3, 4]; EQL-Lite, IJCAI 2007 [7]; SPARQL 1.0, W3C Recommendation 2008 [10]; SPARQL 1.1 (with `FILTER NOT EXISTS`, `MINUS`, subqueries), 2013 [11]; SHACL, 2017 [12].

Against the contemporaries this yields a concrete, checkable contrast rather than a bare assertion of firstness:

- **RQL (2002)** is a declarative, schema-aware RDF query language built around class/property/value navigation; it does not provide a general negation as failure-over-projection (negated subquery) construct for "no known r-successor satisfying φ."
- **RDQL (Jan 2004)** is triple-pattern matching plus filters; its negation is "safe" only, i.e. restricted to clauses with no variables, which is strictly weaker than a negated projected subquery [9].
- **SPARQL 1.0 (2008)** could simulate simple negation as failure via `OPTIONAL` + `!BOUND`, but lacked the general negation and subquery apparatus; **SPARQL 1.1 (2013)** standardized it.
- **EQL-Lite (2007)** is the formal "effective first-order / epistemic query" proposal [7]; it *postdates* nRQL, so nRQL anticipated, in a working engine, the agenda EQL-Lite later formalized.

The defensible statement is therefore:

> By its 2004–2005 publications, nRQL already supported negated projected and negated defined query bodies for OWL/DL querying—several years before SPARQL 1.1 standardized `FILTER NOT EXISTS`, `MINUS`, and subqueries, and before EQL-Lite formalized effective epistemic FO querying. Among *practically available OWL query engines*, nRQL is a strong candidate for the first to combine negation as failure with projection. The underlying *idea* of epistemic/closed-world querying over an open-world KB is older still [5, 6]. We do not claim global firstness over every research prototype, and we do not claim documented influence on later standards.

4 A reduction, with its boundary made explicit

The earlier draft stated an "informal expressivity theorem" asserting that every nonrecursive SHACL Core constraint has an nRQL violation query, *assuming* a faithful substrate mirror of all relevant RDF terms and *assuming* property-path closures are materialized. That statement is true but nearly vacuous: it assumes away precisely the two features (RDF-term-nativeness and unbounded paths) later identified as new. We therefore restate it honestly as a reduction whose hypotheses are the contribution.

Lemma 1 (Reduction modulo mirror, materialization, and a fixed regime)**.** Fix an entailment regime E (e.g. "asserted graph," "RDFS materialization," or "RacerPro DL entailment"). Let G be an RDF/OWL graph represented in a RacerPro KB together with a substrate mirror $M(G)$ that exposes, as substrate nodes/edges/labels, every RDF term, triple/role edge, literal value (with datatype and language tag), and term descriptor (IRI vs. blank vs. literal) that a given shape inspects. Let every property path used be either a bounded path expression or be materialized in $M(G)$ as a finite binary relation. Then for every *nonrecursive* SHACL Core shape S there is a hybrid nRQL query Q_S that returns exactly the focus/value bindings that E-violate S.

Construction. By structural induction on S. Targets become the outer domain (`sh:targetClass C` → concept atom `(?x C)` under E, or a mirrored `rdf:type` query; `sh:targetSubjectsOf/ObjectsOf` → substrate edge atoms). Bounded paths become joins over role/substrate edges, inverses become inverse roles/edges, alternatives become `union`. `sh:minCount 1` → `(neg (project-to ...))`; `sh:minCount n` → existence of n pairwise-distinct value variables or a counting aggregate; `sh:maxCount n` → existence of $n{+}1$ pairwise-distinct values. Value-class/datatype/node-kind/pattern/enumeration → concept, concrete-domain/datatype, or substrate label/predicate atoms. `sh:and/or/not/xone` → `and/union/neg` plus counting for exclusive-or. Qualified value shapes → projected existence/counting over the nested body. `sh:closed` → a query for a mirrored outgoing edge whose predicate label is outside the allowed set. Each step preserves "E-violation," by the induction hypothesis. □

Remark 1 (The boundary is the content)**.** Lemma 1 certifies that the residual novelty of SHACL lives *exactly* in the three hypotheses: (a) the mirror $M(G)$ must already expose RDF-term structure—otherwise term-native components are out of reach (§5.3); (b) unbounded paths must be *materialized*—otherwise they are unreachable in the nonrecursive fragment (§5.2); and (c) the regime E must be *fixed*, because the same surface translation denotes different questions under different regimes (§6). The lemma is not evidence that SHACL added little; it is a map of where it added something.

5 What is genuinely new in SHACL

5.1 Recursive shape conformance is a new semantic object

The earlier draft filed "shape conformance versus violation query" under *soft*, organizational difference. That is right only for the *nonrecursive* fragment, where a shape's satisfaction is captured by a single fixed violation query (Lemma 1). SHACL by design favors shapes that reference other shapes (`sh:node`, `sh:property`, qualified value shapes), and such references may form cycles; the W3C specification deliberately leaves the semantics of recursion open [12].

For *recursive* shapes, conformance is not equivalent to any single nonrecursive query. It requires a *model*—a (partial) assignment of shapes and negated shapes to nodes—and the literature offers several inequivalent choices: the supported/partial-assignment semantics of Corman, Reutter and

Savković [16], a stable-model semantics [17], and a well-founded semantics [18]. The associated decision problems are genuinely hard: validation under these semantics is NP-complete (and NP-hard in the size of the data graph even for a fragment with stratified negation) [16], and satisfiability and containment of recursive SHACL are undecidable for the full language [19]. None of this collapses to "a violation query whose answer set should be empty."

This is the first genuine conceptual addition, and it is a logic-level one: SHACL's conformance relation is a recursively defined, possibly non-stratified, fixpoint object. nRQL's nearest analogue is its stratified negation as failure-with-rules layer, but nRQL's primary semantic object remains an *answer set* for a (compositional, nonrecursive-in-the-body) query, not a global conformance assignment over mutually referential shapes.

5.2 Unbounded regular paths are provably out of the nonrecursive fragment

Bounded paths (sequence, inverse, alternative, fixed-length repetition) reduce to nRQL joins, inverse roles, and unions (Lemma 1). The hard case is `sh:zeroOrMorePath` / `sh:oneOrMorePath` over an *a priori unrestricted* predicate in the raw graph:

```
[ sh:path [ sh:zeroOrMorePath ex:broader ] ; sh:minCount 1 ]
```

This is a regular-path (reachability) query, and it is not expressible by the nonrecursive negation as failure-plus-projection fragment.

Proposition 1 (Unbounded paths exceed nonrecursive nRQL)**.** Let $\mathcal{Q}$ be the class of nRQL queries built from positive atoms, `and`, `union`, `project-to`, and `neg` (negation as failure), *without* recursive rules and *without* relying on a transitivity declaration for the target predicate. There is no $Q \in \mathcal{Q}$ that, for every finite edge-labeled graph, returns exactly the pairs (x, y) connected by an r-path of arbitrary length.

Justification. Queries in $\mathcal{Q}$ are evaluated within (a fragment of) relational algebra / first-order logic over the active domain of known objects. Transitive closure—equivalently, s–t reachability over a binary relation—is not first-order definable on finite structures [20, 21]. Hence no fixed $Q \in \mathcal{Q}$ computes arbitrary-length r-reachability uniformly over all finite graphs. □

nRQL recovers the behaviour only by leaving $\mathcal{Q}$: (i) declaring r a transitive role in the TBox (so DL entailment supplies the closure for positive atoms); (ii) a recursive rule / fixpoint that materializes the closure into the ABox or substrate; or (iii) precomputing the closure into $M(G)$. What SHACL adds is *native* regular-path validation over the raw graph, independent of any OWL role declaration. This is the firmest hard gap in the whole comparison, and Proposition 1 is why—and nRQL's lambda hatch cannot recover it either, being termination-safe (§5.5). What SHACL Core uniquely offers is regular paths as a *declarative* primitive.

The nRQL grammar corroborates this independently. The EBNF for query bodies and ABox query atoms in the RacerPro User's Guide (§4.1.9, pp. 150–153 [1]) provides exactly conjunction (`and`), disjunction (`union`), negation as failure (`neg`), the projection operator (`project-to`), equality/inequality (`same-as`), the `has-known-successor` sugar, and concrete-domain/substrate `:predicate` constraints—*but no transitive-closure or regular-path constructor.* Unbounded reachability is thus absent from the language by construction, not merely awkward to encode; Proposition 1 is the model-theoretic reason the grammar could not have included it within the nonrecursive fragment.

5.3 RDF terms as the primitive validation universe (conditional)

SHACL validates RDF graphs directly: IRIs, blank nodes, literals, datatype IRIs, language tags, and triples are primitive. nRQL's ABox variables range over named individuals; substrate variables range over substrate nodes. The mirror closes much of this gap for OWL KBs, and a hand-built substrate can close more. The residual, genuinely term-level cases—native in SHACL, available in nRQL only if mirrored—include: blank-node-vs-IRI discrimination; exact literal lexical form; language tags as such (not merely the string); `sh:uniqueLang`; and syntactic features of RDF collections or reification. If those features are encoded as substrate labels, nRQL can query them; if not, SHACL sees them and nRQL does not. This is therefore a *conditional* novelty.

5.4 SHACL-SPARQL imports SPARQL algebra and term functions (conditional)

SHACL-SPARQL admits validators written as SPARQL queries; to the extent they rely on SPARQL-specific algebra (property paths, `MINUS`, `VALUES`, `BIND`) or term functions (`isBlank`, `isLiteral`, `datatype`, `lang`), they exceed native nRQL syntax. But the common case—joins, filters, projection, `FILTER NOT EXISTS`—maps onto nRQL conjunction, substrate predicates, projection, and negation as failure. The gap is SPARQL's particular RDF algebra/function vocabulary, not "constraints versus no constraints."

5.5 Escape hatches: nRQL's is deliberately termination-safe

Both languages provide an escape hatch, but they are not of equal power, and an earlier version of this report got the direction wrong. SHACL has SHACL-SPARQL and, in SHACL-JS, *arbitrary JavaScript*—Turing-complete. nRQL has *functional lambda expressions*: a `retrieve1` query whose head contains a `(:lambda ...)` with the `:reject` token defines a server-side filter predicate [1, pp. 146–148]. We previously called this hatch "Turing-complete with full Common Lisp." *That was wrong.* It is RacerPro's *MiniLisp*: a deliberately **termination-safe** sandbox whose interpreter (`source/expressions.lisp` in the RacerPro sources) evaluates a *curated whitelist* of side-effect-free functions and **aborts any recursive call** at runtime ("*call aborted to ensure termination*"). Live testing on RacerPro 2.0 (transcripts in the accompanying repository) confirms the boundary: arithmetic, `format`, list/string operations including substring `search`, `subseq`, `elt`, `reduce`, `some`, bounded `dotimes`, and knowledge-base callbacks (`retrieve`, told-value) are present; `require`, package access (`excl:*`), `loop`, `funcall`, general recursion, and any regular-expression facility are *absent*.

Two consequences follow, and they *sharpen* the residual rather than dissolve it.

- **The hatch does not add full regex.** MiniLisp has substring `search`—so substring `sh:pattern` is expressible, though that is already available declaratively as the substrate `(:predicate (search ...))` atom—but *no* regex engine. A genuine pattern such as `"^[2-8][0-9]*$"` cannot be evaluated, and because recursion is forbidden one cannot hand-roll a general matcher either.
- **The hatch cannot compute transitive closure.** A termination-guaranteed, non-recursive language cannot evaluate unbounded reachability; the unbounded-path residual of Proposition 1 therefore survives the hatch, not merely the declarative fragment.

So the ≈ 5–8% residual of §11—unbounded regular paths and full-regex `sh:pattern`—is *robust*: native to SHACL relative to both nRQL's declarative fragment *and* its termination-safe hatch. The irony is that at the escape-hatch tier it is SHACL's hatches that are the more powerful—SHACL-JS

(arbitrary JavaScript) and SHACL-SPARQL can express both regex and traversal—whereas nRQL deliberately traded that power for a guarantee that query filters cannot diverge. This is the *principled* choice for a *decidable* OWL/DL query engine: the reasoner exists precisely to provide computational guarantees, and a termination-safe filter language preserves them end-to-end, whereas embedding arbitrary JavaScript (SHACL-JS) discards them. The "weaker" hatch is the coherent one.

One honest caveat in the other direction: the MiniLisp whitelist is *extensible*. The same source exposes a server-hook mechanism (`add-server-hook-function`) by which a deployment can register a native Common Lisp function—a `cl-ppcre` regex matcher, say, or a precomputed closure operator—into `+allowed-cl-functions+`, closing the corresponding gap. But that is *extending the language with a native primitive in full Common Lisp*, the analogue of supplying a custom SHACL-JS function or constraint component: an extension story on both sides, not a property of the shipped languages.

Net: the genuine, hatch-independent residual is SHACL's recursive shape-conformance *semantics* (§5.1), its validation-report *model* (§5.5), and its *declarative* regular paths and term/regex primitives—which nRQL matches only by leaving its safe, declarative core.

5.6 Validation reports (organizational)

SHACL makes diagnostics part of the model: validation results carry focus node, value node, result path, message, severity, and source shape [12]. nRQL returns answer tuples or rule consequences; a wrapper can render reports, but the report vocabulary is not part of nRQL. Because standards/tooling are out of scope, this is an organizational rather than expressivity difference—but it is a real conceptual choice.

6 Three semantic traps in the naive translation

The translation table (§7) is sound only once three pitfalls are controlled. These are where "nRQL can express it" most often goes subtly wrong, and the earlier draft underweighted them.

6.1 Epistemic-mode mismatch

The single most important caveat: `(neg (?v C))` means "$C(v)$ is *not entailed* by the KB," which is *neither* "$C(v)$ is false" (classical negation) *nor* "the type triple is absent from the graph" (raw-graph membership). SHACL over a materialized graph asks the last of these. Positive nRQL atoms are open-world and entailment-aware; `neg` is closed-world over the active domain. So the clean-looking row `sh:class C` $\rightarrow$ `(neg (?v C))` denotes three different questions depending on the regime E. Any honest translation must fix E first (Remark 1); the row belongs next to that warning, not in isolation.

6.2 Cardinality without a unique-name assumption

SHACL counts *distinct RDF terms*—a syntactic notion. OWL makes no unique-name assumption: two named individuals may be *inferred equal* (`owl:sameAs`) or *required distinct* (`owl:differentFrom`). So "n+1 pairwise-distinct value variables" for `sh:maxCount n` requires explicit inequality / injective bindings, and even then it counts *known individuals under the chosen equality theory*, not RDF terms. Under `sh:sameAs`-rich data the two counts can diverge. Cardinality is thus one of the *lossy* translations, not a clean one.

6.3 Active domain versus anonymous witnesses

nRQL variables range over named ABox individuals (or mirrored substrate nodes), not over arbitrary model elements. An OWL existential may entail an *anonymous* filler that is no graph term and no named individual; such a witness cannot be bound. This is exactly why `has-known-successor` exists: it asks for a *known* successor, not a merely entailed one [1, pp. 115–116]. SHACL likewise validates graph terms, so the two often agree—but for different reasons, and they diverge when one validates an entailed graph in which existentials have introduced fresh blank nodes.

This trap is worth seeing live (run on RacerPro 2.0; `verification/translation_check.py`). With `parent`$\sqsubseteq \exists$`has-child.`$\top$, assert `alice` a `parent` (no named child) and `bob` a `parent` with a told child. Then `(individual-instance? alice (some has-child top))` returns `t`—`alice` *is* entailed to have a child, so the open-world axiom holds—yet `(retrieve (?x) (?x (has-known-successor has-child)))` returns `bob` only, and the closed-world `minCount` violation query returns `alice`. The same individual satisfies the OWL existential and fails the closed-world check: this is precisely the local-closed-world-over-open-world idiom that the whole comparison turns on, and the reason nRQL positive atoms (entailment) and `NEG` (failure over known objects) must be read with care.

7 Translation patterns (sound under §6)

Table 1 restates the core correspondences, now read *only* subject to a fixed regime E and the caveats above. It is not a claim of syntactic completeness for every corner case; it shows that the constraint *pattern* was semantically available. The principal rows are not merely schematic: `minCount`, `maxCount`, `sh:class` (entailment-aware), `qualifiedMinCount`, `sh:disjoint`, the closed-world `has-known-successor` idiom, and substring `sh:pattern` via the substrate `(:predicate (search ...))` atom were all *executed live* on a RacerPro 2.0 server and returned exactly the expected violations (transcripts: `verification/translation_check.py` and `racer_queries.py`).

Table 1: SHACL Core patterns and nRQL counterparts, modulo a fixed entailment regime.

SHACL pattern	**Semantic reading**	**nRQL/Racer counterpart**
`sh:targetClass C`	Validate nodes of class C.	`(?x C)` under E, or mirrored `rdf:type` query.
`sh:minCount 1`	At least one value on the path.	`(neg (project-to (?x) path))`; the manual's missing-known-successor idiom.
`sh:minCount n`	At least n distinct values.	n pairwise-distinct value variables, or counting aggregate. *See §6.2.*
`sh:maxCount n`	At most n distinct values.	Violation: $n{+}1$ pairwise-distinct values; injective bindings / inequality. *Lossy without UNA.*
`sh:class C`	Each value conforms to C.	Violation: value v with `(neg (?v C))`. *Meaning depends on E; see §6.1.*

SHACL pattern	Semantic reading	nRQL/Racer counterpart
`sh:datatype d`	Value is a literal of datatype d.	Concrete-domain/datatype atoms, or mirrored literal markers.
`sh:nodeKind`	IRI/blank/literal term-kind.	Substrate term-kind labels if mirrored; not an OWL fact.
`sh:pattern`	Lexical regex check.	Substrate string predicates (manual shows substring search).
`sh:languageIn`, `sh:uniqueLang`	Language-tag constraints.	Only if language tags are mirrored as data; otherwise native to SHACL.
`sh:hasValue`, `sh:in`	Required/enumerated values.	Equality/`same-as` or substrate literal equality; negation as failure for absence.
`sh:equals`, `sh:disjoint`	Compare value sets of two paths.	Difference via projection + negation as failure; disjointness as a shared value.
`sh:lessThan(OrEquals)`	Cross-property comparison.	Concrete-domain/substrate binary numeric/string predicates.
`sh:qualifiedMin/MaxCount`	Count values meeting a nested shape.	Project/count over the nested body; `NEG` for missing qualified values.
`sh:and/or/not/xone`	Boolean shape combinators.	`and`/`union`/`neg`; xor via counting satisfied alternatives.
`sh:closed true`	No outgoing properties except allowed.	Query a mirrored outgoing edge whose predicate label is not allowed; needs predicate labels in the substrate.
Bounded `sh:path` (seq/inv/alt)	Navigate finite graph paths.	Joins with intermediate variables; inverse roles/edges; `union`.
`sh:zero/oneOrMorePath`	Unbounded reachability.	**Out of the nonrecursive fragment (Prop. 1).** Only via transitive role, recursive rule closure, or materialization.

7.1 Annotation and ontology-metadata governance

Because of the mirror, an SHACL-SPARQL shape requiring annotations on subclass axioms has a hybrid nRQL analogue once axiom nodes, sources, targets, properties, and annotations are mirrored:

Listing 3: Schematic nRQL version over a mirrored OWL substrate (marker names depend on the encoding).

```
(retrieve (?*ax ?*x ?*y)
```

```
(and (?*ax ?*x (:owl-annotated-source))
     (?*ax *rdfs:subClassOf (:owl-annotated-property))
     (?*ax ?*y (:owl-annotated-target))
     (neg
       (project-to (?*ax)
         (and (?*ax ?*p (:owl-annotation-relationship))
              (?*p RequiredAnnotationKind))))))
```

This is the point that most often surprises modern readers: ontology-metadata validation was *not* uniquely enabled by SHACL [1, pp. 141–143].

8 Where nRQL was stronger than SHACL Core

The comparison is not one-directional. For OWL-aware constraints nRQL was often the more powerful instrument:

- **Native DL reasoning inside constraints.** Positive atoms exploit entailed class/role membership; a single query can mix entailment, classical negation, negation as failure, substrate model-checking, and concrete-domain predicates. SHACL Core validates a graph and pushes entailment outside the shape language, into a preprocessing/regime choice [12].
- **Hybrid semantic atoms.** ABox and substrate variables bind in parallel, integrating OWL conditions with graph/data predicates in one query [1, pp. 135–137]—tighter than SHACL Core's separation of validation and reasoning.
- **Non-monotonic update rules.** nRQL rules have query-body antecedents and generalized ABox consequents, can create individuals, and are non-monotonic via negation as failure [1, pp. 125–127]; the reference manual confirms `firerule` as the rule analogue of `retrieve` [2, pp. 160–161]. SHACL Core has no update semantics; SHACL Advanced Features adds rules later [13].

9 What SHACL still cannot do, twenty years on

The comparison so far asked what SHACL *added*. The converse is just as telling, and sharper than the prose of §8 suggests: a substantial part of the nRQL/RacerPro repertoire has no SHACL counterpart even today, because SHACL is by design a validator over a fixed graph, not a reasoner. To make the point concrete rather than rhetorical, every row of Table 2 was *executed live* on a RacerPro 2.0 server; the transcripts are in the accompanying repository (`verification/reasoning_demos.py`). For good measure the first two rows were also reproduced from an actual *loaded OWL file*: RacerPro parsed an ontology defining `DogOwner` $\equiv$ `Person` $\sqcap \exists$`owns.Dog`, classified `DogOwner` $\sqsubseteq$ `PetOwner` (never asserted, from `Dog` $\sqsubseteq$ `Pet`), and realized the lone individual—asserted only as a `Person` owning a `Dog`—as both a `DogOwner` and a `PetOwner` (`verification/owl_classification.py`).

Table 2: Reasoning services with no SHACL Core counterpart, each run live on RacerPro 2.0.

Service	**nRQL / RacerPro (executed live)**	**SHACL**
Membership by *inference*	`betty` asserted only as a `woman` with a child; `(individual-instance? betty mother)` → `t` for `mother`≡`woman`⊓∃`has-child.human`.	Validates the asserted/materialized graph; cannot infer types from a definition.
Logical (un)satisfiability	`robin` a `man` and a `woman` (disjoint) ⇒ `(abox-consistent?)` → `nil`.	No notion of contradiction; a graph merely conforms or not.
Inferred subsumption	`dog`⊑`animal` ⇒ `(concept-subsumes? animal-owner dog-owner)` → `t`, never asserted.	No TBox, no subsumption.
Role-axiom reasoning	`ancestor` transitive, only `c`→`d` on the chain asserted; query for `d`'s ancestors → `a, b, c`.	Paths are syntactic over the asserted graph; no transitivity/inverse/sub-role axioms.
Cardinality without UNA	`at-most 1 has-child` with two *named* children: consistent, until `(different-from k1 k2)` flips it to `nil`.	Counts distinct RDF terms; no equality reasoning.
Non-monotonic rules	`firerule` with a negation as failure antecedent derives new assertions (e.g. `has-grandchild`).	Core: none; Advanced Features adds (monotonic) rules.
Query consistency	`(query-consistent-p Q)` → `nil` for an unsatisfiable query body—guaranteed empty on *every* ABox.	Shape satisfiability is undecidable in general [19]; no built-in service.
Query subsumption	`(query-entails-p Q1 Q2)` → `t`: every answer of `Q1` is an answer of `Q2` in every model.	Shape containment is likewise a hard research problem [19]; no built-in service.

The last two rows are the most striking. SHACL *satisfiability* and *containment*—can a shape ever be satisfied? does one shape subsume another?—are the subject of recent research and are *undecidable* in the general recursive case [19]. RacerPro shipped `query-consistent-p` and `query-entails-p` as routine (if deliberately *incomplete*, hence terminating) nRQL services two decades earlier: nRQL could reason about its *own queries*, not just answer them. The honest framing is not that SHACL is deficient—it is a validation language, meant to be paired with a separate reasoner or materializer—but that the integrated package *reasoning + closed-world query + rules + query metareasoning*, in one decidable system, is something SHACL does not provide and was never meant to.

10 A scoped headline, with no fabricated percentage

The earlier draft summarized the result as "≈ 80%." We withdraw the number. It was disclaimed repeatedly in that draft, which is itself a sign that it carried rhetorical weight it could not support, and in an archival or priority context it is the first detail a skeptical reader would attack. The defensible statement is qualitative:

> For the recurring family of *OWL-centric, finite, closed-world* integrity checks—required properties, cardinalities, qualified value constraints, type/datatype/string checks, disjointness/equality, annotation and ontology-metadata governance, and absence-of-counterexample constraints—*and modulo a fixed entailment regime*, nRQL plus the mirror substrate was already expressively adequate by 2005. The residual is concentrated in recursive shape conformance (§5.1), native unbounded paths (§5.2), RDF-term primitives (§5.3), and the standardized conformance/report model.

We replace the point estimate with a measured *band*. Coding the W3C SHACL test suite against Table 1 (§11) yields the headline we endorse:

> Roughly **73–84% of SHACL Core constraint features have a direct nRQL counterpart** (and ≈92% are expressible once the substrate mirror is granted), while the **provably-native residual is only** ≈5%—**entirely unbounded or recursive property paths**—rising to a best single estimate of ≈8% once `sh:pattern` regular expressions (which RacerPro cannot express, having no regex engine) are added.

The width of the band is set by how much RDF-term and lexical structure one assumes mirrored (§11); that dependence on assumptions is the honest replacement for a single number. Crucially, the part that is *provably* beyond nonrecursive nRQL (Proposition 1) does not move with those assumptions: it is the unbounded-path category. A second, smaller gap—`sh:pattern` regular expressions—also does not depend on mirroring, since RacerPro's concrete domain offers only `string=/string<>` and the substrate a substring `search`; and like the paths it is *not* closed by nRQL's lambda hatch, which is a deliberately termination-safe "MiniLisp" with no regex engine (§5.5)—it is SHACL's own SPARQL/JS hatches that are Turing-complete. So the residual is robust to the escape-hatch objection; what ultimately remains native to SHACL is its conformance semantics, report model, and declarative regular-path/term primitives. We carry out the coding for the W3C SHACL test suite in §11.

11 Empirical coding of the W3C SHACL test suite

Rather than leave the "large majority" qualitative, we coded the official W3C SHACL test suite—the `data-shapes-test-suite` that accompanies the 2017 Recommendation [12]—against Table 1.

11.1 Method

The unit of analysis is one *main* test file (manifests and `-data`/`-shapes` include files excluded; constraint components unioned across a test's includes). Each `core/` test is assigned the feature it isolates—the suite is organized by constraint component, so the filename names the feature under test—and then a bucket:

- **E** (expressible): a direct nRQL/RacerPro counterpart exists under a fixed regime and a faithful mirror (Lemma 1);

- **C** (conditional/lossy): expressible only if specific RDF-term structure is mirrored, or with a documented §6 caveat (cardinality without UNA; lexical checks via substrate string predicates; term kind; language tags; closed shapes);
- **N** (native residual, relative to *declarative* nRQL—see §5.5 for the escape-hatch tier): outside the nonrecursive negation as failure-plus-projection fragment (unbounded or recursive paths; recursion) *or* beyond RacerPro's declarative predicate vocabulary (`sh:pattern` regular expressions);
- **M** (meta/reporting): not a constraint at all (`sh:deactivated`, `sh:message`, `sh:severity`)—the organizational category of §5.5.

Two feature families are bucketed by *actual content*, not filename: path tests are N iff the shapes graph uses `sh:zeroOrMorePath`/`sh:oneOrMorePath`; `sh:pattern` tests are C iff the pattern is a literal substring with no flags (expressible via the substrate `search` predicate) and N otherwise, since RacerPro has no regex engine (User's Guide pp. 63–64, 137). The coding script and a per-file audit CSV accompany this report so the coding can be replicated and contested. The `core/complex` composite tests (2) and the `sparql/` tests (23; the SHACL-SPARQL extension, conditional by construction) are reported separately, not folded into the Core totals.

11.2 Results

Across the 95 coded `core/` tests:

Bucket	Meaning	n	%
E	Expressible (direct nRQL counterpart)	68	71.6
C	Conditional / lossy (mirror or §6 caveat)	15	15.8
N	Native vs. *declarative* nRQL (paths + regex)	7	7.4
M	Meta / reporting (organizational)	5	5.3
	Total	95	100.0

Table 3: Coding of the W3C SHACL Core test suite against Table 1.

Restricting to the 83 genuine *constraint* tests (excluding 7 target-selection and 5 meta tests) gives $E = 61$ (73.5%), $C = 15$ (18.1%), $N = 7$ (8.4%). The seven native tests split into two concrete, non-overlapping reasons. *Four* are unbounded or recursive property paths—`path-zeroOrMore-001`, `path-oneOrMore-001`, `path-complex-001` (which uses `sh:zeroOrMorePath`), and `path-unused-001` (whose shape is a recursive `sh:zeroOrMorePath`); these are the *provably* native core (4.8%), exactly the category Proposition 1 isolates, and the nRQL grammar confirms the language has no path-closure operator. The other *three* are `sh:pattern` tests whose patterns are genuine regular expressions or carry flags (`node/pattern-001` `"^[2-8][0-9]*$"`, and the case-insensitive `node/pattern-002` and `property/pattern-002`); RacerPro has no regex engine—neither in its declarative fragment (only `string=`/`string<>` and substring `search`) *nor* in its lambda/`:reject` hatch, the termination-safe "MiniLisp" of §5.5—so these three are native to SHACL robustly. The single substring `pattern` (`property/pattern-001`, `"Joh"`) remains in C, since substring search *is* available. So all seven N tests are native relative to both nRQL's declarative fragment and its escape hatch; only a deployment that registered a native regex primitive (§5.5) would reduce the regex three. We also executed the two core idioms live on RacerPro 2.0: the `minCount` violation query returned exactly the value-less individual, and the `sh:class` query returned only the offending parent—correctly

passing a parent whose child is merely a subclass instance, confirming the DL-entailment claim. Transcripts accompany the report.

11.3 Sensitivity, and the fate of "80%"

The C bucket is where coder judgment bites, so we report a band, not a point. After the regex reclassification, C comprises one substring `pattern` (1), lexical length checks (`minLength`, `maxLength`; 4), term-kind/language checks (`nodeKind`, `languageIn`, `uniqueLang`; 6), closed shapes (2), and `maxCount` (2). Moving the length, `maxCount`, substring-`pattern`, and closed tests to E—each expressible with documented substrate/concrete-domain machinery—raises direct expressibility to $\approx 84\%$; conversely, treating every RDF-term-native and closed-shape feature as unreachable because nothing is mirrored raises the native residual to $15/83 \approx 18\%$. Hence

$$\text{direct-expressible } E \in [73.5\%,\ \approx 84\%], \qquad \text{native residual } N \in [\approx 5\%,\ \approx 18\%],$$

with a best single estimate of $N \approx 8\%$ (paths plus `sh:pattern` regexes) and a *provable* core of 4.8% (paths alone). The earlier draft's "80%" still lands inside the E band. This vindicates it as a *plausible midpoint* while confirming the methodological point of §10: the honest object is a band whose width is fixed by explicit mirroring assumptions, not a single number—and the part that is *provably* new is small ($\approx 5\%$) and entirely concentrated in unbounded paths. Notably, the manual cut both ways: verifying the concrete domain first *lowered* the optimistic ceiling (from $\approx 88\%$ to $\approx 84\%$) by exposing a regex gap, then the lambda/`:reject` escape hatch (§5.5) *raised* it again by closing that gap—a reminder that primary sources cut in both directions, and that all of these figures describe the *declarative* fragment, not the escape-hatch tier where the expressivity residual vanishes.

11.4 Threats to validity

1. **Coverage, not frequency.** A conformance suite weights each component by test count, not by how often it occurs in deployed shapes. This measures feature-level expressibility *coverage*, not deployment *share*; a usage-frequency study over real shapes graphs would be a complementary and different experiment.

2. **Recursion is under-exercised.** The 2017 Core suite barely tests recursive shapes (they surface mainly in the `core/complex` SHACL-of-SHACL test). The genuine hardness of recursion (§5.1) is a decision-problem fact—NP-hardness and undecidability [16, 19]—not a feature-coverage fact, so the measured residual *understates* the conceptual novelty of recursive conformance.

3. **Single automated coder.** One coding scheme was applied; no second coder or inter-rater agreement was computed. The per-file CSV is published so the coding can be audited.

12 Property graphs: a correction

An earlier version of this report declined the property-graph comparison on the ground that "the substrate differs from modern property graphs ... modern property graphs treat edges as first-class entities with their own property maps, which the substrate does not." *That was factually wrong*, and we correct it here. The RacerPro data substrate creates nodes *and* edges with the `data-node`/`data-edge` macros, each carrying a structured, queryable label, retrieved by `node-label`/`edge-label` [1, pp. 137–139]. Edge labels are first-class key–value property maps, exactly as in a property graph.

Figure 1 (a live RacerPro 2.0 session) makes this concrete. A financial-transaction graph is built with accounts as nodes carrying `:amount`, `:currency`, and `:number` properties, and a `deposit` edge carrying its own `:amount` and `:currency`:

```
(data-node acc1 ((:amount 1000.00) (:currency $) (:number 10039)) account)
(data-node acc2 ((:amount 19000.00) (:currency $) (:number 2090)) account)
(data-edge acc1 acc2 ((:amount 9900.00) (:currency $)) deposit)
```

and queried with nRQL predicates over node *and* edge properties—"$ accounts with balance >10000 that received a withdrawal of >5000", where the amount bound is a predicate on the edge's property map:

```
(retrieve (?x ?y)
  (and (?x account)
       (?*x ((:currency $) (:amount (:predicate (> 10000)))))
       (?x ?y withdrawal)
       (?*y ?*x ((:currency $) (:amount (:predicate (> 5000)))))))
```

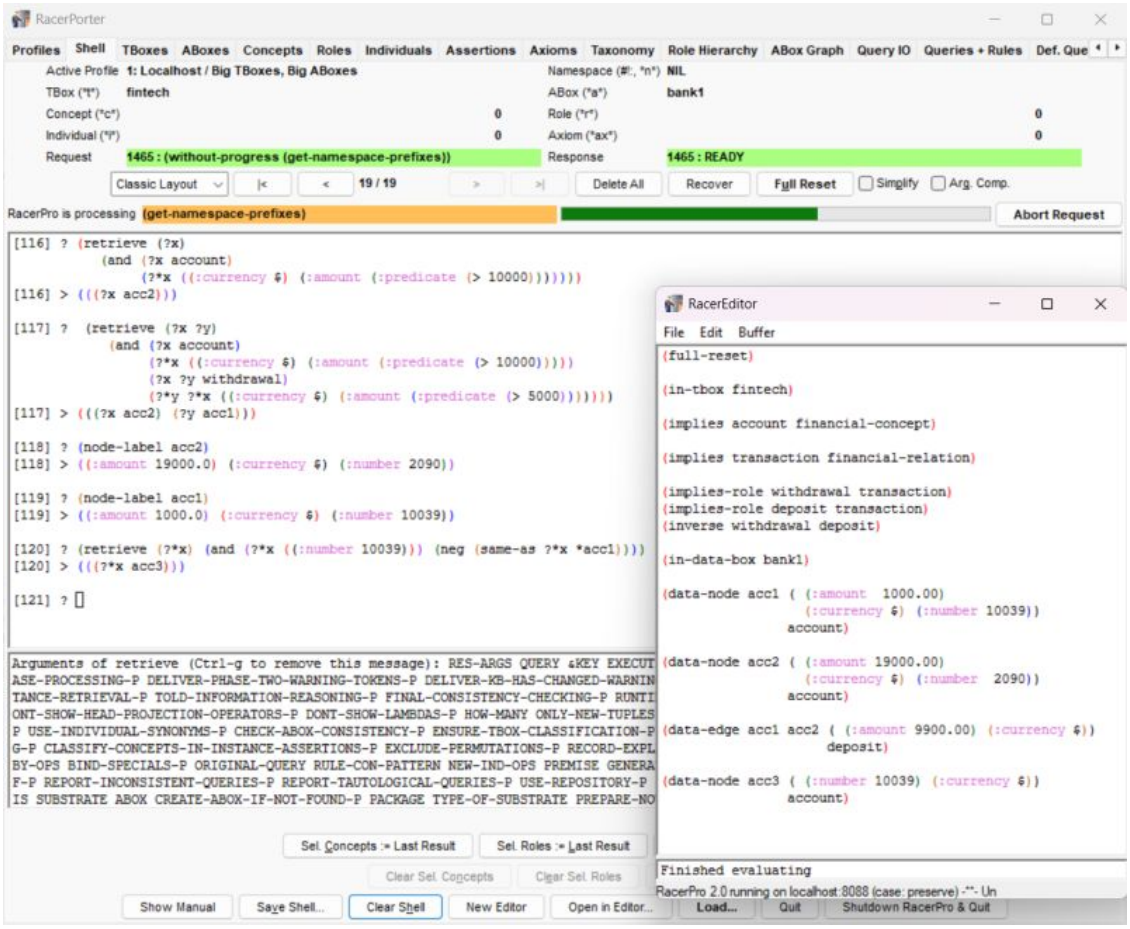

Figure 1: A property graph built and queried in RacerPro 2.0 (RacerPorter). Nodes `acc1`–`acc3` and the `deposit` edge carry key–value property maps; nRQL filters on node *and* edge properties and traverses the `deposit`/`withdrawal` (inverse) edge, while node types (`account`) are OWL/DL concepts and edge labels are DL roles.

Two things are notable. First, this is a bona fide property graph—typed nodes and typed directed

edges, both with key–value property maps, queried directly—and it predates the *term* "property graph." Second, and more distinctively, it is a property graph *loosely coupled to a description-logic reasoner*: the node type `account` is a TBox concept (`account` $\sqsubseteq$ `financial-concept`) and the edge labels `deposit`/`withdrawal` are TBox roles related by `inverse`, so a hybrid nRQL query mixes property-graph pattern matching with OWL/DL concept and role reasoning, inverses, and negation as failure. **We are aware of no prior or contemporary system that combined a queryable property graph with a tableau DL reasoner this way.** The substrate also answers the motivation directly: attaching `:amount`/`:currency` to the `deposit` *relationship* is awkward in RDF or DL (it forces reification or n-ary workarounds), which is exactly the shortcoming property graphs were meant to fix. This is not a paper exercise: the entire session of Figure 1 was reproduced live on RacerPro 2.0, returning the answers shown (transcript in the accompanying repository).

12.1 Priority, calibrated as in §3

The attributed/labeled-graph database *model* is much older than 2005: Kuper and Vardi's Logical Data Model (1984) [23], the GOOD and hypernode models (1990) [24], and especially Güting's *GraphDB* (1994) [25]—which already modelled graphs with attributes on nodes and edges, with a query language, motivated by transportation networks—are recognizably graph-with-attributes models, surveyed by Angles and Gutierrez [22]. The branded model and the term came later: the Neo4j labeled-property-graph model was sketched around 2000 and released publicly around 2007, "property graph" was coined by Rodriguez and Neubauer in 2010 [26], and formal/standard treatments are later still (Angles' formal model, 2018; ISO GQL and SQL/PGQ, 2023–2024). RacerPro's 2005 substrate therefore sits *between* the academic graph-data-model tradition and the productized property-graph wave—an independent reinvention of a recurring idea.

So the calibrated claim is neither "RacerPro invented property graphs" (the model predates it by two decades) nor "RacerPro influenced Cypher/GQL" (no citation trail; these emerged from the graph-database and application world [27]). It is the defensible middle: RacerPro's data substrate is an early (2005), working, *reasoning-coupled* property-graph store with edge-attribute querying—legitimate, and currently uncited, prior art for the combination of property graphs with description-logic reasoning, exhibiting the same convergent evolution that recurs throughout this history rather than any documented lineage.

13 Answer to the main question

> **SHACL did not introduce the ability to express closed-world integrity constraints over OWL data, required-property and qualified-cardinality constraints, annotation-governance constraints, or most datatype/string checks. Those were already expressible in nRQL via negation as failure, projection, rules, concrete-domain/datatype predicates, and the mirror/substrate layer [3, 4, 1], and the underlying epistemic-query idea is older still [5, 6].**

What SHACL did add, in decreasing order of how "hard" the novelty is:

1. **Recursive shape conformance** as a first-class, possibly non-stratified fixpoint semantics, with NP-hard validation and undecidable satisfiability/containment in general [16, 17, 19]. This does not reduce to a single nonrecursive violation query.

2. **Native unbounded regular paths**, provably outside the nonrecursive negation as failure-plus-projection fragment (Prop. 1; [20, 21]).

3. **An RDF-term-native validation universe** (node kind, lexical form, language tags), conditional on what a nRQL substrate mirrors (§5.3).

4. **SPARQL algebra/term functions** when SHACL-SPARQL is used—conditional, since the common fragment still maps to hybrid nRQL.

5. **A standardized conformance/validation-report model**—an organizational rather than expressivity gain.

What was *not* new: negated subqueries and universal closed-world queries, which nRQL realized via `NEG` over `PROJECT-TO` years before SPARQL 1.1 standardized them.

14 Conclusion

A historically fair account should not say that SHACL was the first practical way to impose closed-world constraints on OWL data, nor that SPARQL 1.1 first made negated-subquery universal checks possible. nRQL supplied the essential mechanisms—negation as failure, projection, entailment-aware atoms, rules, concrete-domain predicates, aggregation, complex TBox queries, and a hybrid substrate—and the mirror data substrate made OWL annotations and told values queryable, dissolving the false "domain ABox versus ontology metadata" boundary. nRQL deserves explicit recognition as prior art for negated projected subqueries in a practical OWL query engine, predating SPARQL 1.1 [3, 4].

The central difference is orientation. nRQL is a reasoner-integrated query-and-rule language that *can be used to* validate, whose primary object is an answer set. SHACL is a validation language whose primary object is a conformance relation between data and shapes. That orientation buys SHACL two things that are not mere repackaging—recursive conformance semantics and native regular-path validation—plus RDF-term primitives and a report model. It does not buy it the closed-world constraint idea, which the description-logic tradition and nRQL already had. What remains genuinely open, in both directions, is *influence*: the conceptual debt is clear at the level of prior art and anticipation, but no citation trail establishes that SPARQL 1.1, SHACL, or Cypher built directly on nRQL.

A Manual evidence, summarized

The source points from the uploaded manuals most relevant to the comparison:

1. nRQL described historically as an expressive DL query language with conjunctive queries, variables over named objects, negation as failure, projection, group-by, aggregation [1, p. 4].

2. Feature summary: atoms combine with `and`/`union`/`neg`/`project-to`; positive variables bind by entailment; negation as failure and classical negation; concrete-domain and datatype/annotation retrieval; projection and aggregation; hybrid substrate queries; the autoepistemic characterization [1, pp. 81–84].

3. negation as failure section: classical negation vs. negation as failure; missing-known-child idiom with `project-to` and `has-known-successor` [1, pp. 113–120].

4. Rules: query-body antecedents, generalized ABox consequents, individual creation, non-monotonicity [1, pp. 125–127]; `firerule` as the analogue of `retrieve` [2, pp. 160–161].

5. Substrate: node/edge-labeled directed graph associated with an ABox; hybrid queries over both [1, pp. 133–137].

6. Mirror data substrate: substrate objects for ABox and OWL elements; OWL annotation/-datatype markers; annotation/told values queryable as substrate nodes [1, pp. 137–143].

7. `NEG` over `project-to` and over defined queries: the negated-subquery idiom for closed-world universal constraints [1, pp. 119–122].

References

[1] Racer Systems GmbH & Co. KG. *RacerPro User's Guide Version 2.0*. October 24, 2012. https://github.com/lambdamikel/Racer/tree/master/doc. (Page numbers in this report are the printed page numbers of this edition; syntax was verified against §4.1.9, pp. 150–153.)

[2] Racer Systems GmbH & Co. KG. *RacerPro Reference Manual Version 1.9*. October 12, 2010. https://github.com/lambdamikel/Racer/tree/master/doc.

[3] Volker Haarslev, Ralf Möller, and Michael Wessel. Querying the Semantic Web with Racer + nRQL. In *Proc. KI-2004 International Workshop on Applications of Description Logics (ADL'04)*, Ulm, Germany, September 2004.

[4] Michael Wessel and Ralf Möller. A High Performance Semantic Web Query Answering Engine. In *Proc. 2005 International Workshop on Description Logics (DL 2005)*, CEUR-WS Vol. 147, 2005.

[5] Francesco M. Donini, Maurizio Lenzerini, Daniele Nardi, Werner Nutt, and Andrea Schaerf. An Epistemic Operator for Description Logics. *Artificial Intelligence*, 100(1–2):225–274, 1998.

[6] Francesco M. Donini, Daniele Nardi, and Riccardo Rosati. Description Logics of Minimal Knowledge and Negation as Failure. *ACM Transactions on Computational Logic*, 3(2):177–225, 2002.

[7] Diego Calvanese, Giuseppe De Giacomo, Domenico Lembo, Maurizio Lenzerini, and Riccardo Rosati. EQL-Lite: Effective First-Order Query Processing in Description Logics. In *Proc. 20th International Joint Conference on Artificial Intelligence (IJCAI 2007)*, pages 274–279, 2007.

[8] Gregory Karvounarakis, Sofia Alexaki, Vassilis Christophides, Dimitris Plexousakis, and Michel Scholl. RQL: A Declarative Query Language for RDF. In *Proc. 11th International World Wide Web Conference (WWW 2002)*, pages 592–603, 2002.

[9] Andy Seaborne. *RDQL — A Query Language for RDF*. W3C Member Submission, 9 January 2004. https://www.w3.org/submissions/2004/SUBM-RDQL-20040109/.

[10] Eric Prud'hommeaux and Andy Seaborne, editors. *SPARQL Query Language for RDF*. W3C Recommendation, 15 January 2008. https://www.w3.org/TR/rdf-sparql-query/.

[11] Steve Harris and Andy Seaborne, editors. *SPARQL 1.1 Query Language*. W3C Recommendation, 21 March 2013. https://www.w3.org/TR/sparql11-query/.

[12] Holger Knublauch and Dimitris Kontokostas, editors. *Shapes Constraint Language (SHACL)*. W3C Recommendation, 20 July 2017. https://www.w3.org/TR/shacl/.

[13] Holger Knublauch, Dimitris Kontokostas, et al. *SHACL Advanced Features*. W3C Working Group Note, 8 June 2017. https://www.w3.org/TR/shacl-af/.

[14] Eric Prud'hommeaux, Jose Emilio Labra Gayo, and Harold Solbrig. Shape Expressions: An RDF Validation and Transformation Language. In *Proc. 10th International Conference on Semantic Systems (SEMANTICS 2014)*, pages 32–40, 2014.

[15] Holger Knublauch. *SPIN — Modeling Vocabulary*. W3C Member Submission, 22 February 2011. https://www.w3.org/Submission/spin-modeling/.

[16] Julien Corman, Juan L. Reutter, and Ognjen Savković. Semantics and Validation of Recursive SHACL. In *Proc. 17th International Semantic Web Conference (ISWC 2018)*, LNCS 11136, pages 318–336, 2018.

[17] Medina Andreşel, Julien Corman, Magdalena Ortiz, Juan L. Reutter, Ognjen Savković, and Mantas Šimkus. Stable Model Semantics for Recursive SHACL. In *Proc. The Web Conference 2020 (WWW 2020)*, pages 1570–1580, 2020.

[18] Cem Okulmus et al. SHACL Validation under the Well-founded Semantics. In *Proc. 21st International Conference on Principles of Knowledge Representation and Reasoning (KR 2024)*, 2024.

[19] Paolo Pareti, George Konstantinidis, Fabio Mogavero, and Timothy J. Norman. SHACL Satisfiability and Containment. In *Proc. 19th International Semantic Web Conference (ISWC 2020)*, LNCS 12506, pages 474–493, 2020.

[20] Alfred V. Aho and Jeffrey D. Ullman. The Universality of Data Retrieval Languages. In *Proc. 6th ACM Symposium on Principles of Programming Languages (POPL 1979)*, pages 110–120, 1979.

[21] Leonid Libkin. *Elements of Finite Model Theory*. Springer, 2004.

[22] Renzo Angles and Claudio Gutierrez. Survey of Graph Database Models. *ACM Computing Surveys*, 40(1), Article 1, 2008.

[23] Gabriel M. Kuper and Moshe Y. Vardi. A New Approach to Database Logic (The Logical Data Model). In *Proc. 3rd ACM SIGACT-SIGMOD Symposium on Principles of Database Systems (PODS 1984)*, 1984; journal version *ACM Transactions on Database Systems* 18(3):379–413, 1993.

[24] Mark Levene and Alexandra Poulovassilis. The Hypernode Model and its Associated Query Language. In *Proc. 5th Jerusalem Conference on Information Technology (JCIT)*, pages 520–530, 1990. See also the GOOD model: Marc Gyssens, Jan Paredaens, and Dirk Van Gucht, A Graph-Oriented Object Database Model, In *Proc. 9th ACM Symposium on Principles of Database Systems (PODS 1990)*, pages 417–424.

[25] Ralf Hartmut Güting. GraphDB: Modeling and Querying Graphs in Databases. In *Proc. 20th International Conference on Very Large Data Bases (VLDB 1994)*, pages 297–308, 1994.

[26] Marko A. Rodriguez and Peter Neubauer. Constructions from Dots and Lines. *Bulletin of the American Society for Information Science and Technology*, 36(6):35–41, 2010.

[27] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. Cypher: An Evolving Query Language for Property Graphs. In *Proc. 2018 International Conference on Management of Data (SIGMOD 2018)*, pages 1433–1445, 2018.